

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

Coding Challenge

Code of Conduct:

I P.Sai Ganesh Reddy(192525111)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

P.Sai Ganesh Reddy

Annexure A

Short Answer Test

1. Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.
2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.
3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.
4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.
5. Write a Python program to implement both KNN and RBF models on the same dataset. Compare their predictions for given test inputs. Analyze differences in computation, accuracy, and output behavior, and display results clearly for better comparison.
6. Write a Python program to implement the **Decision Tree algorithm** for classification. Use a small dataset and construct the tree using information gain or Gini index. Allow prediction for test instances and display accuracy. Visualize the tree structure and analyze how feature selection impacts classification performance.
7. Develop a Python program to implement a **Naïve Bayes classifier** for a given dataset. Compute prior and conditional probabilities, and classify a test instance. Display predicted class labels and evaluate performance using accuracy. Compare results with and without Laplace smoothing and analyze its impact on classification.
8. Write a Python program to implement **Logistic Regression** for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate affects convergence and performance.
9. Create a Python program to implement the **K-Means clustering algorithm**. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

10. Write a Python program to implement a simple **Genetic Algorithm** to optimize a mathematical function. Initialize a population, perform selection, crossover, and mutation operations, and evolve solutions over generations. Display fitness values and best solution, and analyze how parameters influence convergence speed and solution quality.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1, Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

Aim:

To implement the K-Nearest Neighbour algorithm using Euclidean distance and classify a test instance.

Algorithm:

1. Start.
2. Store the training dataset with feature values and class labels.
3. Read the value of **K** from the user.
4. Read the test instance.
5. Calculate the Euclidean distance between the test instance and every training point.
6. Sort the distances in ascending order.
7. Select the **K nearest neighbours**.
8. Count the class labels of the selected neighbours.
9. Assign the class with the highest number of votes to the test instance.
10. Display the nearest neighbours and predicted class.
11. Stop.

Training Data:

1 2 A

2 3 A

3 4 A

6 7 B

7 8 B

8 9 B

Test Instance:

5 6

Enter K: 3

Output:

```
... Enter K: 3
Nearest Neighbors: [(1.4142135623730951, 'B'), (2.8284271247461903, 'A'), (2.8284271247461903, 'B')]
Predicted Class: B
```

Result:

Thus, the KNN algorithm was successfully implemented using Euclidean distance and the test instance was classified.

2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

Aim

To implement Locally Weighted Regression using a Gaussian kernel and compare its prediction with standard Linear Regression.

Algorithm

1. Start.
2. Create a small dataset.
3. Select a query point.
4. Calculate the distance between each training point and the query point.
5. Calculate weights using a Gaussian kernel.
6. Construct the weighted matrices.
7. Calculate the locally weighted regression prediction.
8. Calculate the standard linear regression prediction.
9. Display both predictions.
10. Stop.

Input

Dataset:

$X = [1, 2, 3, 4, 5]$

$Y = [2, 4, 6, 8, 10]$

Enter query point: 3.5

Output

```
Enter query point: 3.5
LWR Prediction: 7.0
Linear Regression Prediction: 7.0
```

Result

Thus, Locally Weighted Regression was successfully implemented using a Gaussian kernel and compared with standard linear regression.

3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.

Aim

To implement a simple Radial Basis Function network using Gaussian activation functions for regression.

Algorithm

1. Start.
2. Create the training dataset.
3. Select RBF centres.
4. Calculate Gaussian activation for each input.
5. Construct the activation matrix.
6. Calculate output weights using the pseudo-inverse.
7. Predict the output for test inputs.
8. Display predicted values.
9. Stop.

Input

Training Data:

$X = [1, 2, 3, 4, 5]$

$Y = [2, 4, 6, 8, 10]$

Centers = [1, 3, 5]

Width = 1.0

Enter test value: 3

Output

```
Enter test value: 3
RBF Predicted Output: 5.95
Centers: [1. 3. 5.]
width: 1.0
```

Result

Thus, the RBF network was successfully implemented using Gaussian activation functions.

4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.

Aim

To implement a basic Case-Based Learning system that retrieves the most similar previous case using Euclidean distance.

Algorithm

1. Start.
2. Store previous cases with their solutions.
3. Read a new query.
4. Calculate the Euclidean distance between the query and every stored case.
5. Find the case having the minimum distance.
6. Retrieve its solution.
7. Display the closest case and solution.
8. Stop.

Input

Stored Cases:

[1, 1] -> Low

[2, 2] -> Medium

[5, 5] -> High

[8, 8] -> Very High

Enter query values: 4 4

Output

```
Enter query values: 4.4
Closest Case: [5, 5]
Distance: 0.6
Retrieved Solution: High
```

Result

Thus, the Case-Based Learning system was successfully implemented and the solution of the most similar case was retrieved.

5. Write a Python program to implement both KNN and RBF models on the same dataset. Compare their predictions for given test inputs. Analyze differences in computation, accuracy, and output behavior, and display results clearly for better comparison.

Aim

To implement KNN and RBF models on the same dataset and compare their predictions.

Algorithm

1. Start.
2. Create a common dataset.
3. Implement KNN using Euclidean distance.
4. Select the nearest neighbours and predict the class.
5. Implement the RBF model using Gaussian functions.
6. Train the RBF model using the training data.
7. Predict the output for the same test inputs.
8. Display and compare the predictions.
9. Stop.

Input

X = [1, 2, 3, 6, 7, 8]

Y = [0, 0, 0, 1, 1, 1]

Enter test value: 7

Output

```
Enter test value: 7
KNN Prediction: 1
RBF Prediction: 1
```

Result

Thus, KNN and RBF models were successfully implemented and their predictions were compared on the same dataset.

6. Write a Python program to implement the **Decision Tree algorithm** for classification. Use a small dataset and construct the tree using information gain or Gini index. Allow prediction for test instances and display accuracy. Visualize the tree structure and analyze how feature selection impacts classification performance.

Aim

To implement a Decision Tree classifier using the Gini index and evaluate its classification accuracy.

Algorithm

1. Start.
2. Create the dataset.
3. Split the dataset into training and testing data.
4. Create a Decision Tree classifier using the Gini index.
5. Train the model using the training data.
6. Predict the classes of the test data.
7. Calculate accuracy.
8. Visualize the decision tree.
9. Stop.

Input

Training Data:

[1,1] -> 0
[1,2] -> 0
[2,1] -> 0
[2,2] -> 0
[8,8] -> 1
[8,9] -> 1
[9,8] -> 1
[9,9] -> 1

Enter test values: 8 8

Output

```
Predicted Labels: [0 1]
Actual Labels: [0, 1]
Accuracy: 1.0
Enter test values: 8 8
Predicted Class: 1
```

Result

Thus, the Decision Tree classifier was successfully implemented using the Gini index and its classification accuracy was evaluated.

7. Develop a Python program to implement a **Naïve Bayes classifier** for a given dataset. Compute prior and conditional probabilities, and classify a test instance. Display predicted class labels and evaluate performance using accuracy. Compare results with and without Laplace smoothing and analyze its impact on classification.

Aim

To implement a Naïve Bayes classifier and compare classification with and without Laplace smoothing.

Algorithm

1. Start.
2. Create the training dataset.
3. Calculate the prior probability of each class.
4. Calculate conditional probabilities for each feature.
5. Calculate the probability of each class for the test instance.
6. Select the class with the highest probability.
7. Repeat the calculation using Laplace smoothing.
8. Compare the results.
9. Stop.

Input

Training Data:

[0,0] -> 0

[0,1] -> 0

[1,0] -> 1

[1,1] -> 1

[1,1] -> 1

[0,0] -> 0

Enter test values: 1 0

Output

```
Enter test values: 1 0
Prediction without Laplace smoothing: 1
Prediction with Laplace smoothing: 1
```

Result

Thus, the Naïve Bayes classifier was successfully implemented and the effect of Laplace smoothing was compared.

8. Write a Python program to implement **Logistic Regression** for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate affects convergence and performance.

Aim

To implement Logistic Regression for binary classification using gradient descent.

Algorithm

1. Start.
2. Create the training dataset.
3. Initialize weights and bias.
4. Calculate the linear combination of inputs.
5. Apply the sigmoid function.
6. Calculate the error.
7. Update weights and bias using gradient descent.
8. Repeat for the specified number of iterations.
9. Classify test instances.
10. Calculate accuracy.
11. Stop.

Input

Training Data:

$X = [1, 2, 3, 4, 5]$

$Y = [0, 0, 1, 1, 1]$

Enter learning rate: 0.1

Enter test value: 3

Output

```
Enter learning rate: 3
Enter test value: 3
Sigmoid Output: 0.9807
Predicted Class: 1
Accuracy: 100.0 %
```

Result

Thus, Logistic Regression was successfully implemented using gradient descent, and the sigmoid output, predicted class, and accuracy were obtained.

9. Create a Python program to implement the **K-Means clustering algorithm**. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

Aim

To implement the K-Means clustering algorithm and display the final clusters and centroids.

Algorithm

1. Start.
2. Create the dataset.
3. Select the number of clusters K.
4. Initialize the cluster centroids.
5. Assign each point to the nearest centroid.
6. Calculate new centroids.
7. Repeat the assignment and centroid update steps until convergence.
8. Display the final clusters and centroids.
9. Visualize the clusters.
10. Stop.

Input

Dataset:

[1,1]
[1,2]
[2,1]
[2,2]
[8,8]
[8,9]
[9,8]
[9,9]

Enter number of clusters: 2

Output

```
Enter number of clusters: 2
Cluster Labels: [0 0 0 0 1 1 1 1]
Centroids:
[[1.5 1.5]
 [8.5 8.5]]
```

Result

Thus, the K-Means clustering algorithm was successfully implemented and the data points were grouped into two clusters.

10. Write a Python program to implement a simple **Genetic Algorithm** to optimize a mathematical function. Initialize a population, perform selection, crossover, and mutation operations, and evolve solutions over generations. Display fitness values and best solution, and analyze how parameters influence convergence speed and solution quality.

Aim

To implement a simple Genetic Algorithm to maximize a mathematical function using selection, crossover, and mutation.

Algorithm

1. Start.
2. Initialize a random population.
3. Calculate the fitness of each chromosome.
4. Select the chromosomes with higher fitness.
5. Perform crossover to generate new solutions.
6. Perform mutation on selected chromosomes.
7. Form a new population.
8. Repeat the process for several generations.
9. Display the best solution and fitness value.
10. Stop.

Input

Population Size: 10

Generations: 20

Mutation Rate: 0.1

Fitness Function: $f(x) = x^2$

Search Range: 0 to 31

Output

```
Best Solution: 31
Best Fitness: 961
Generations: 20
```

Result

Thus, the Genetic Algorithm was successfully implemented using selection, crossover, and mutation, and the best solution was obtained.